MINI PROJECT REPORT ON

"Campus Clubs - College Club Management System"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Pratham Tapadia (T23, 2401120)

Anirudh Vineet (T23, 2401124)

Komal Yannam (T23, 2401127)

SUBJECT

Agile Software Development and DevOps

SUPERVISOR

Prof. Sanober Shaikh



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050

UNIVERSITY OF MUMBAI [Academic Year: 2026–27]

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2026-27

Certificate

This is to certify that the Mini Project entitled **"Campus Clubs - College Club Management System"** is a bonafide work of **Pratham Tapadia, Anirudh Vineet and Komal Yannam** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **"Bachelor of Engineering"** in **"Artificial Intelligence and Data Science".**

Prof. Sanober Shaikh

(Supervisor)

Abstract

Campus Clubs is a web-based college club management system that brings club discovery, membership, events and announcements into a single application. Visitors can browse a searchable club directory, students can register, join clubs and RSVP to events, club administrators manage their own club's members, events and announcements, and site administrators create and remove clubs. The application is written in Python with the Flask framework and the SQLAlchemy ORM, uses SQLite for local development and PostgreSQL in the Docker deployment, and is served by Gunicorn. Access to every management action is decided by role-based checks, and RSVPs are rejected for non-members, for events that have already started and for events that have reached their capacity.

Beyond the application, the project applies an Agile and DevOps toolchain. The application exposes Prometheus metrics at /metrics covering HTTP traffic, user and club activity and database totals; Prometheus scrapes them and evaluates four alert rules, and Grafana shows them on a pre-provisioned Club Manager dashboard. Docker Compose starts the web application, PostgreSQL, Prometheus and Grafana together. A GitHub Actions workflow runs Ruff and pytest, validates the monitoring and Compose configuration, smoke-tests the full stack and publishes the container image from the main branch. Work is tracked in the Jira project AM and referenced from commit messages. The supplied automated test suite contains 39 tests, all of which passed when it was run for this report.

TABLE OF CONTENTS

Content **Page No.**

Abstract **3**

List of Figures **4**

1. Introduction **5**

1.1 Background 5

1.2 Problem Statement 5

2. Implementation **6**

2.1 Technologies Used 6

2.2 Code Structure 7

2.3 Database Design 8

2.4 Club Management and Access-Control Logic 10

2.5 Monitoring and Delivery Pipeline 11

3. Results and Discussion **13**

3.1 Result 13

3.2 Discussion 17

4. Conclusion **19**

4.1 Conclusion 19

4.2 Future Scope 19

5. References **20**

LIST OF FIGURES

Fig. 2.1 Entity-relationship diagram derived from app/models.py 9

Fig. 3.1 Campus Clubs home page 14

Fig. 3.2 Club directory with search and category filters 14

Fig. 3.3 Jira backlog of the ASD MPR project 15

Fig. 3.4 GitHub repository AnirudhVineet/ASD-MPR 15

Fig. 3.5 GitHub Actions CI/CD workflow runs 16

Fig. 3.6 Docker Desktop showing the club-manager:local image 16

Fig. 3.7 Grafana Club Manager dashboard 17

1. Introduction

1.1 Background

Student clubs are where much of campus life happens, yet the information that keeps them running is usually scattered. Membership lists sit in spreadsheets, event announcements travel through messaging groups, and students who want to join a club often cannot tell which clubs exist, who runs them or what is happening next. Club organisers, in turn, have no single place to see who belongs to the club, who plans to attend an event or whether an event has room left.

A centralised club management platform addresses this by giving each of these needs one consistent home. Campus Clubs is a web application in which visitors can browse and search the club directory, students can create an account, join clubs and RSVP to events, and each club has administrators who maintain the club page, its members, its events and its announcements. A site administrator oversees the whole directory by creating and deleting clubs.

The project is also an exercise in Agile software development and DevOps practice. The application is packaged with Docker and started together with PostgreSQL, Prometheus and Grafana by Docker Compose. A GitHub Actions workflow lints and tests every push and pull request to the main branch, work items are tracked in the Jira project AM, and commit messages carry the Jira keys so that code changes can be traced back to the planned work.

1.2 Problem Statement

The problem is the lack of one shared, role-aware place for club information. Without it, students cannot reliably discover clubs, organisers cannot manage members and events with consistent rules, and nobody can tell whether the system itself is healthy. Events can be over-subscribed or advertised to people who are not members, and club-management actions are easy to perform by the wrong person if permissions are not enforced in one place.

The system therefore aims to provide the following capabilities:

1. Provide a public club directory with name search and category filter, and club pages that list upcoming events.
2. Let students register, log in, join and leave clubs, and see a personal dashboard of their clubs, upcoming events, RSVPs and announcements.
3. Enforce role-based access for visitors, club members, club administrators and site administrators.
4. Let club administrators edit their club, promote, demote and remove members, create, edit and delete events, view attendee lists and publish announcements.
5. Apply RSVP rules: only members may RSVP, RSVPs are blocked for past events, and RSVPs are blocked once an event’s capacity is reached.
6. Expose Prometheus metrics, a provisioned Grafana dashboard and alert rules so that traffic, latency, errors and campus activity can be observed.
7. Provide a repeatable, containerised build and run process, verified by a GitHub Actions pipeline, with work traced through Jira.

Chapter 2

Implementation

2.1 Technologies Used

Component	Technology	Port	Role
Web application	Python 3.12, Flask 3.0.3	8000	Server-rendered pages, blueprints, application factory
Application server	Gunicorn 23.0.0	8000	Serves the Flask app (1 worker, 8 threads)
ORM	Flask-SQLAlchemy 3.1.1, SQLAlchemy 2.0.35	none	Models, relationships and queries
Sessions and forms	Flask-Login 0.6.3, Flask-WTF 1.2.1	none	Login sessions and CSRF protection
Database (Docker)	PostgreSQL 16 (postgres:16-alpine), psycopg 3.2.3	5432 (internal only)	Persistent storage in the db service
Database (local)	SQLite	none	Default for local development and tests
Metrics exporter	prometheus-client 0.21.0	8000 (/metrics)	HTTP, authentication and business metrics
Prometheus	prom/prometheus v2.54.1	9090	Scrapes metrics, evaluates alert rules
Grafana	grafana/grafana 11.2.2	3000	Pre-provisioned Club Manager dashboard
CI/CD	GitHub Actions	none	Lint, test, validate, smoke-test, publish image
Code quality	Ruff 0.6.9, pytest 8.3.3	none	Linting and automated tests
Tracking	Git, GitHub, Jira (project AM)	none	Version control and work items

Table 2.1: Technology stack and ports (host ports as mapped in docker-compose.yml; web, Prometheus and Grafana ports can be changed through .env)

- **User interface:** server-rendered Jinja templates with a single stylesheet (static/style.css) and a small script (static/app.js) for the mobile navigation toggle and dismissible flash messages. Pages share a base layout and reusable macros.
- **Application logic:** Flask blueprints for the public pages, authentication, clubs, events and metrics, created by an application factory in app/__init__.py.
- **Data layer:** SQLAlchemy models on SQLite for local development and on PostgreSQL in Docker Compose. The connection string is read from the DATABASE_URL environment variable.
- **Observability:** prometheus-client exposes /metrics on the same port as the application. Prometheus scrapes it every 15 seconds and Grafana reads from Prometheus.
- **Containerisation:** a single Dockerfile based on python:3.12-slim runs the application as a non-root user, and docker-compose.yml starts the web, database, Prometheus and Grafana services.

- **Continuous integration:** one GitHub Actions workflow (.github/workflows/ci-cd.yml) with four jobs, described in Section 2.5.
- **Version control and tracking:** Git and GitHub, with commit messages and branches tagged with Jira keys such as AM-36.

2.2 Code Structure

The repository is organised around one Flask package, with the infrastructure files in the repository root.

- **app/** – the Flask application: __init__.py (application factory, error handlers), config.py, extensions.py, models.py, auth.py, clubs.py, events.py, main.py, metrics.py and cli.py (the create-admin command).
- **app/templates/ and app/static/** – Jinja templates grouped by feature (auth/, clubs/, events/, plus base, dashboard, index and error pages) and the stylesheet and script.
- **tests/** – the pytest suite: test_auth.py, test_clubs.py, test_events.py, test_main.py, test_metrics.py and test_templates.py, with shared fixtures in conftest.py and helpers.py.
- **monitoring/** – prometheus/prometheus.yml and alerts.yml, and grafana/provisioning (data source and dashboard provider) with the dashboard JSON club-manager.json.
- **Dockerfile and docker-compose.yml** – the container image and the four-service stack.
- **.github/workflows/ci-cd.yml** – the GitHub Actions workflow.
- **pyproject.toml, requirements.txt, requirements-dev.txt, .env.example, wsgi.py, README.md** – tool configuration, pinned dependencies, environment-variable template, the Gunicorn entry point and the project documentation.

The application uses five blueprints. The main blueprint serves the home page, the dashboard and the /healthz endpoint. The auth blueprint handles registration, login and logout. The clubs blueprint (URL prefix /clubs) covers the directory, club pages, membership and announcements. The events blueprint covers event listing, creation, editing, deletion and RSVPs. The metrics blueprint serves /metrics. Table 2.2 lists the routes.

Route	Method	Access	Purpose
/	GET	Public	Home page with upcoming events and newest clubs
/clubs/	GET	Public	Club directory with ?q (name search) and ?category filter
/clubs/<id>	GET	Public	Club details, upcoming events, announcements
/clubs/new	GET, POST	Site admin	Create a club
/clubs/<id>/delete	POST	Site admin	Delete a club
/clubs/<id>/edit	GET, POST	Club admin or site admin	Edit club name, category and description
/clubs/<id>/join, /leave	POST	Logged in	Join or leave a club
/clubs/<id>/members	GET	Club admin or site admin	List the club’s members

Route	Method	Access	Purpose
/clubs/<id>/members/<uid>/ remove, /role	POST	Club admin or site admin	Remove a member; promote or demote
/clubs/<id>/announcements	POST	Club admin or site admin	Publish an announcement
/clubs/<id>/announcements/ <aid>/edit	GET, POST	Club admin or site admin	Edit an announcement
/clubs/<id>/announcements/ <aid>/delete	POST	Club admin or site admin	Delete an announcement
/events	GET	Public	All upcoming events
/clubs/<id>/events/new	GET, POST	Club admin or site admin	Create an event
/events/<id>	GET	Public	Event details; attendee list shown to managers
/events/<id>/edit	GET, POST	Club admin or site admin	Edit an event
/events/<id>/delete	POST	Club admin or site admin	Delete an event
/events/<id>/rsvp, /rsvp/cancel	POST	Logged in	RSVP (members only) or cancel an RSVP
/register, /login	GET, POST	Public	Account creation and login
/logout	POST	Logged in	End the session
/dashboard	GET	Logged in	My clubs, upcoming events, RSVPs and announcements
/healthz	GET	Public	Status ok, or 503 if the database is unreachable
/metrics	GET	Public	Prometheus metrics

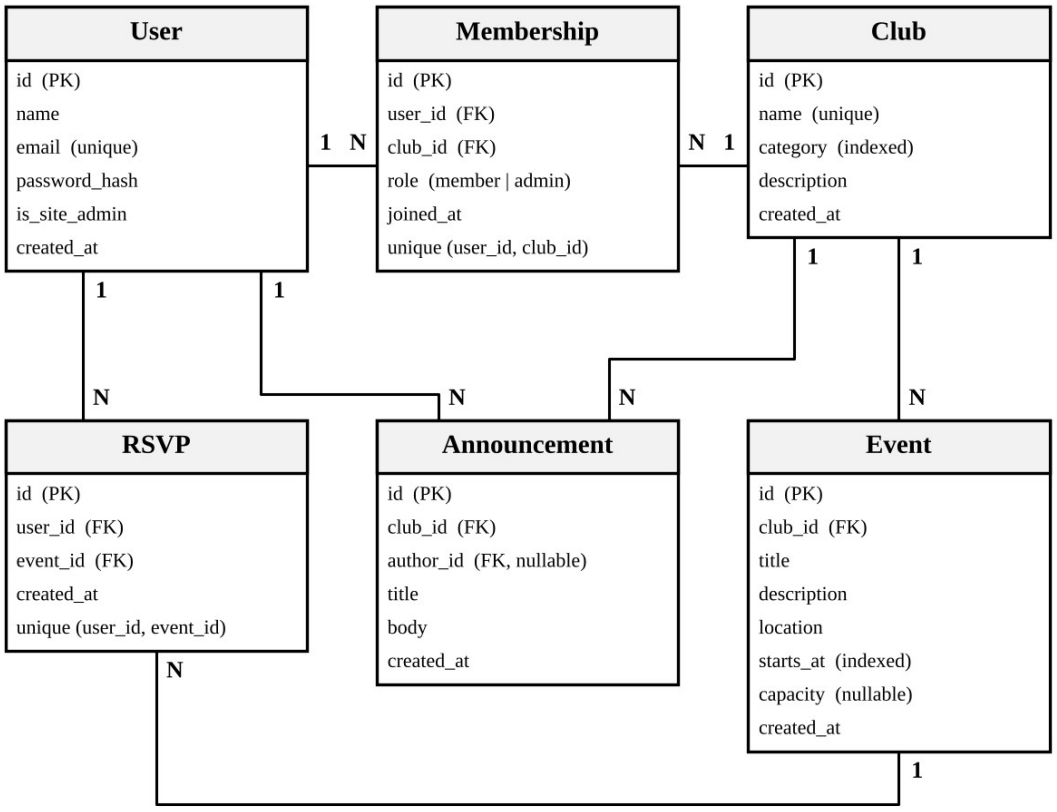
Table 2.2: Application routes and the access they require

2.3 Database Design

The data model is defined in app/models.py with six SQLAlchemy entities. User and Club are linked many-to-many through Membership, which also stores the member’s role in that club. Events and announcements belong to a club, and an RSVP links a user to an event. Table 2.3 summarises the entities and Fig. 2.1 shows their relationships, which were derived directly from the model definitions.

Entity	Columns	Constraints and relationships
User	id, name, email, password_hash, is_site_admin, created_at	email unique and indexed; has many Memberships (cascade delete-orphan); passwords are stored as hashes
Club	id, name, category, description, created_at	name unique; category indexed; has many Memberships, Events and Announcements (all cascade delete-orphan)
Membership	id, user_id, club_id, role, joined_at	Foreign keys to User and Club; unique (user_id, club_id); role is member or admin
Event	id, club_id, title, description, location, starts_at, capacity, created_at	Foreign key to Club; starts_at indexed; capacity is optional (null means unlimited); has many RSVPs (cascade)
RSVP	id, user_id, event_id, created_at	Foreign keys to User and Event; unique (user_id, event_id)
Announcement	id, club_id, author_id, title, body, created_at	Foreign key to Club; author_id references User and may be null

Table 2.3: Database entities (from app/models.py)



User and Club are in a many-to-many relationship resolved by Membership (which carries the role).
Deleting a Club cascades to its Memberships, Events and Announcements; deleting an Event cascades to its RSVPs.

Fig. 2.1: Entity-relationship diagram derived from app/models.py

The database is selected by configuration. config.py reads DATABASE_URL and falls back to sqlite:///clubs.db; connections use pool_pre_ping. In Docker Compose the variable points to the PostgreSQL service through the psycopg driver, and the data is kept in the named volume

pgdata. Tables are created at start-up with db.create_all(); the project does not use a migration tool. The test configuration uses an in-memory SQLite database.

2.4 Club Management and Access-Control Logic

Four levels of access exist. A visitor is not logged in. A student is a logged-in user. A club member is a student with a Membership row for that club, and a club admin is a member whose Membership role is admin. A site admin is a user with is_site_admin set, which can only be done with the flask create-admin command. Access is checked in a few shared helpers: login_required for authentication, User.can_manage(club), which is true for a site admin or for a club admin of that club, require_manager(club), which aborts with 403 otherwise, and the site_admin_required decorator for club creation and deletion. Table 2.4 summarises the resulting permissions.

Action	Visitor	Member	Club admin	Site admin
Browse and search clubs, view clubs and events	Yes	Yes	Yes	Yes
Join or leave a club	No	Yes	Yes	Yes
RSVP to an event	No	Own clubs	Own clubs	Only if a member
Edit club details	No	No	Own club	Any club
Promote, demote or remove members	No	No	Own club	Any club
Create, edit, delete events; view attendees	No	No	Own club	Any club
Publish, edit, delete announcements	No	No	Own club	Any club
Create or delete a club	No	No	No	Yes

Table 2.4: Role permissions as implemented in the route handlers

Membership. Joining creates a Membership with role member; the unique constraint and an existence check make the action idempotent, and leaving deletes the row. Managers can remove a member and change a member’s role to member or admin; any other role value is rejected with HTTP 400. When a site admin creates a club, an existing user’s email can be given so that this user is immediately made the club admin.

Events and announcements. Event forms require a title and a valid start time in the format YYYY-MM-DDTHH:MM, and the optional capacity must be a positive whole number. Announcements require a title and a message. Club forms require a name that is unique ignoring case and a category from a fixed list of eight categories.

RSVP validation. The rsvp handler applies the following checks in order and stops at the first that fails:

1. The user must be a member of the event’s club; otherwise the message “Join the club to RSVP to its events” is shown.
2. The event must not have started (starts_at earlier than the current time is rejected).
3. The user must not already have an RSVP for the event.
4. The event must not be full: attendee_count must be below capacity when a capacity is set.

Only when all checks pass is an RSVP row created. Cancelling removes it. Registration additionally requires a name, an email containing “@” and a password of at least eight characters, and login redirects only to relative ?next paths to avoid open redirects. All forms are protected by Flask-WTF CSRF tokens, and session cookies are HttpOnly with SameSite=Lax.

2.5 Monitoring and Delivery Pipeline

The application exports the metrics in Table 2.5. The six business gauges are produced by a custom collector that queries the database when Prometheus scrapes, so they remain correct however many application processes are running. HTTP metrics are labelled with the URL rule (for example /clubs/<int:club_id>) instead of the raw path, which keeps label cardinality bounded.

Metric	Type	Labels	Description
http_requests_total	Counter	method, endpoint, status	Total HTTP requests
http_request_duration_seconds	Histogram	method, endpoint	Request latency (buckets from 5 ms to 5 s)
clubapp_users	Gauge	none	Registered users
clubapp_clubs	Gauge	none	Clubs
clubapp_memberships	Gauge	none	Club memberships
clubapp_upcoming_events	Gauge	none	Events that have not started
clubapp_event_rsvps	Gauge	none	Event RSVPs
clubapp_announcements	Gauge	none	Announcements
clubapp_logins_total	Counter	result	Login attempts (success, failure)
clubapp_registrations_total	Counter	none	New registrations
clubapp_membership_changes_total	Counter	action	Membership changes (join, leave, remove)
clubapp_rsvps_total	Counter	action	RSVP actions (rsvp, cancel)

Table 2.5: Prometheus metrics exported by the application

Prometheus scrapes the clubapp job (web:8000, path /metrics) and its own endpoint every 15 seconds, retains data for 15 days and loads the alert rules in Table 2.6. No Alertmanager is configured, so alerts are visible in the Prometheus interface but are not sent anywhere.

Alert	Condition	For	Severity
ClubAppDown	up{job="clubapp"} == 0	1 m	critical
ClubAppHighErrorRate	5xx share of requests, 5-minute rate, above 5%	5 m	warning
ClubAppSlowResponses	p95 of http_request_duration_seconds above 1 s	10 m	warning
NoNewRegistrations	No increase of clubapp_registrations_total in 24 h	30 m	info

Table 2.6: Prometheus alert rules (monitoring/prometheus/alerts.yml)

Grafana is provisioned from files: a Prometheus data source (uid prometheus, http://prometheus:9090) and a dashboard provider that loads club-manager.json into a Club

Manager folder, with editing from the interface disabled. The Club Manager dashboard refreshes every 30 seconds and has three groups of panels: Health (app up, requests per second, 5xx ratio, p95 latency, logins in 24 hours), Campus activity (users, clubs, memberships, upcoming events, RSVPs, announcements) and Traffic (request rate by endpoint, latency percentiles, responses by status, user activity per hour and a membership trend).

Docker Compose defines four services. db runs postgres:16-alpine with a health check; web is built from the Dockerfile, publishes port 8000 and waits until the database is healthy; prometheus and grafana publish ports 9090 and 3000. Secrets (SECRET_KEY, the database password and the Grafana admin password) come from a .env file created from .env.example, and Compose refuses to start if they are missing. The image runs Gunicorn with one worker and eight threads, so that the in-process counters stay consistent, and has a health check on /healthz.

The GitHub Actions workflow runs on pushes and pull requests to main and can also be started manually. Table 2.7 lists its four jobs.

Job	**What it does**
1. Lint & test	Installs requirements-dev.txt on Python 3.12, then runs ruff check . and pytest -v
2. Validate monitoring & compose config	Runs promtool check config (which includes the alert rules), checks that the dashboard JSON parses, and runs docker compose config
3. Docker Compose smoke test	After jobs 1 and 2: starts the full stack, calls /healthz, the home page, /clubs/ and /metrics, runs create-admin against PostgreSQL, confirms that Prometheus sees the app target as up and that Grafana has loaded the dashboard
4. Publish image to GHCR	Only for pushes to main, after the smoke test: builds the image and pushes it to GitHub Container Registry with the tags latest and sha-<short>

Table 2.7: GitHub Actions jobs (.github/workflows/ci-cd.yml)

Work is planned as Jira work items in the project AM (ASD MPR). Branches, commits and pull requests carry the issue key, for example AM-36-update-readme and “AM-28 Add edit announcement feature for club admins”, so that GitHub for Jira can link code changes to issues. Development was carried out on the branch Praddy and other feature branches and merged into main through pull requests.

Chapter 3

Results and Discussion

3.1 Result

The results below are separated by the kind of evidence behind them: observed in the supplied screenshots, verified by running the supplied automated tests, or implemented in the source code but not shown in a screenshot.

- **Public pages (screenshots):** the home page presents the Campus Clubs hero, an Upcoming events section (empty at the time of capture) and the newest clubs (Fig. 3.1). The club directory has a name search box and category filters for Academic, Arts & Culture, Community Service, Cultural, Media, Sports, Technology and Other, and lists one club, IETE, in the Technology category with one member (Fig. 3.2).
- **Student and club-admin features (code and tests):** registration, login, joining and leaving clubs, the dashboard, member management, event management, attendee lists, announcements and the RSVP rules are implemented in app/auth.py, clubs.py, events.py and main.py and are exercised by the automated tests. No screenshot of the student dashboard or of the event and RSVP pages was supplied, so these screens are not shown in this report.
- **Automated tests (executed for this report):** ruff check reported no problems and all 39 pytest tests passed on the supplied source, in a separate environment running Python 3.13 (Table 3.1). This is a functional result only; no coverage percentage was measured.
- **Monitoring (screenshot):** the Club Manager dashboard in Grafana shows App up = 1, 0.0326 requests per second, a p95 latency of 4.75 ms, and 2 users, 1 club, 1 membership, 0 upcoming events, 0 RSVPs and 0 announcements (Fig. 3.7). The 5xx ratio and Logins (24h) panels show No data, meaning no samples were available for those series in the selected window. The traffic panel legend lists the endpoint labels /, /clubs/<int:club_id>, /healthz, /static/<path:filename> and unmatched, which confirms that URL rules are used as labels.
- **Container image (screenshot):** Docker Desktop lists the local image club-manager:local, 241.89 MB with 21 layers, built on a Python 3.12 base image (Fig. 3.6). The image has not been analysed by Docker Scout in the screenshot, so no vulnerability result is reported.
- **Pipeline (screenshot):** the GitHub Actions page lists 11 runs of the CI/CD workflow, and all runs visible in Fig. 3.5 show a successful status, including runs for pull requests from the Praddy branch and for merges into main.
- **Traceability (screenshots):** the repository page shows 22 commits and 5 branches, and commit messages carry Jira keys such as AM-7, AM-8, AM-17, AM-28, AM-30, AM-32 and AM-36 (Fig. 3.4). The Jira backlog of the project ASD MPR lists work items AM-25, AM-28, AM-30, AM-31 and AM-32 (Fig. 3.3).

Screenshots

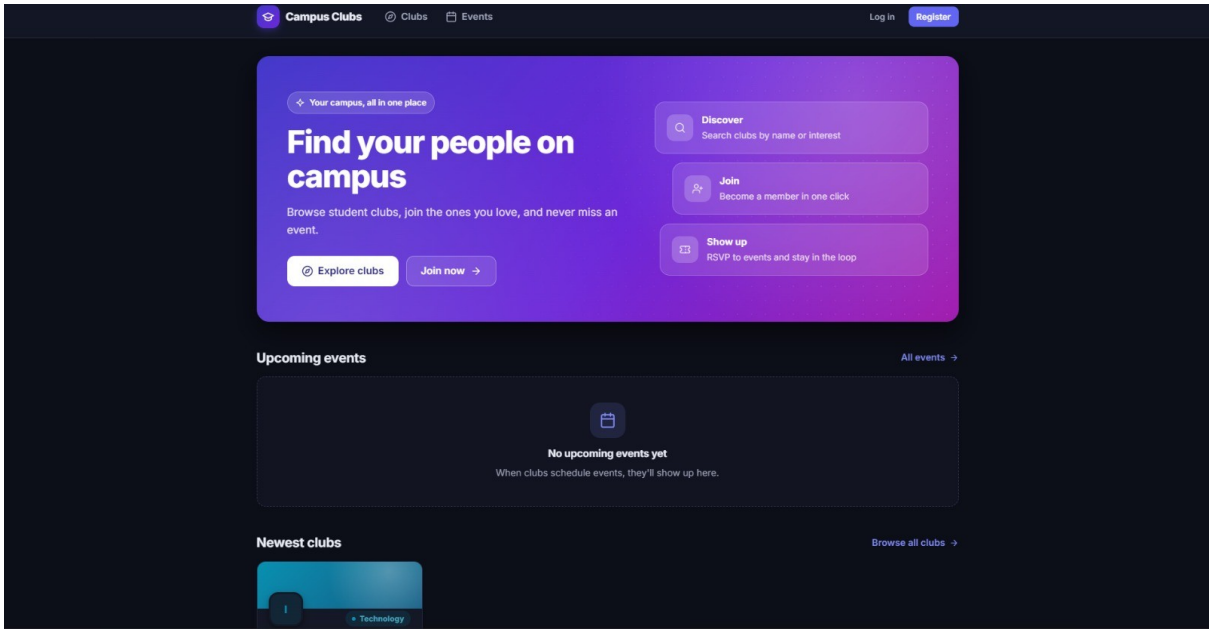


Fig. 3.1: Campus Clubs home page

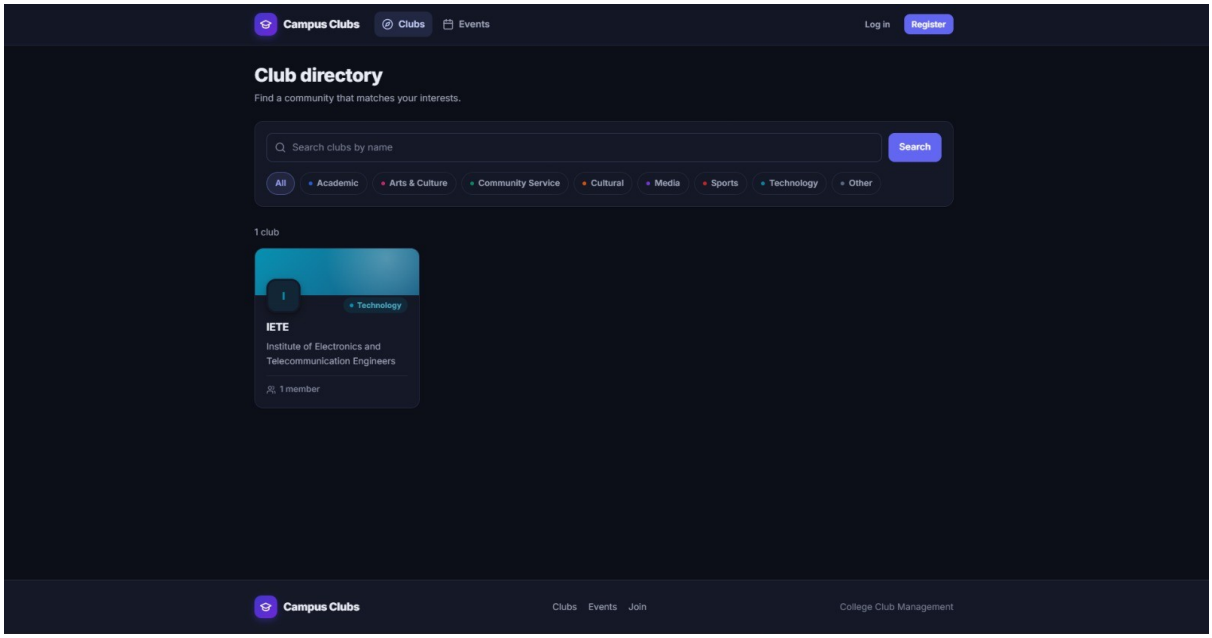


Fig. 3.2: Club directory with search and category filters

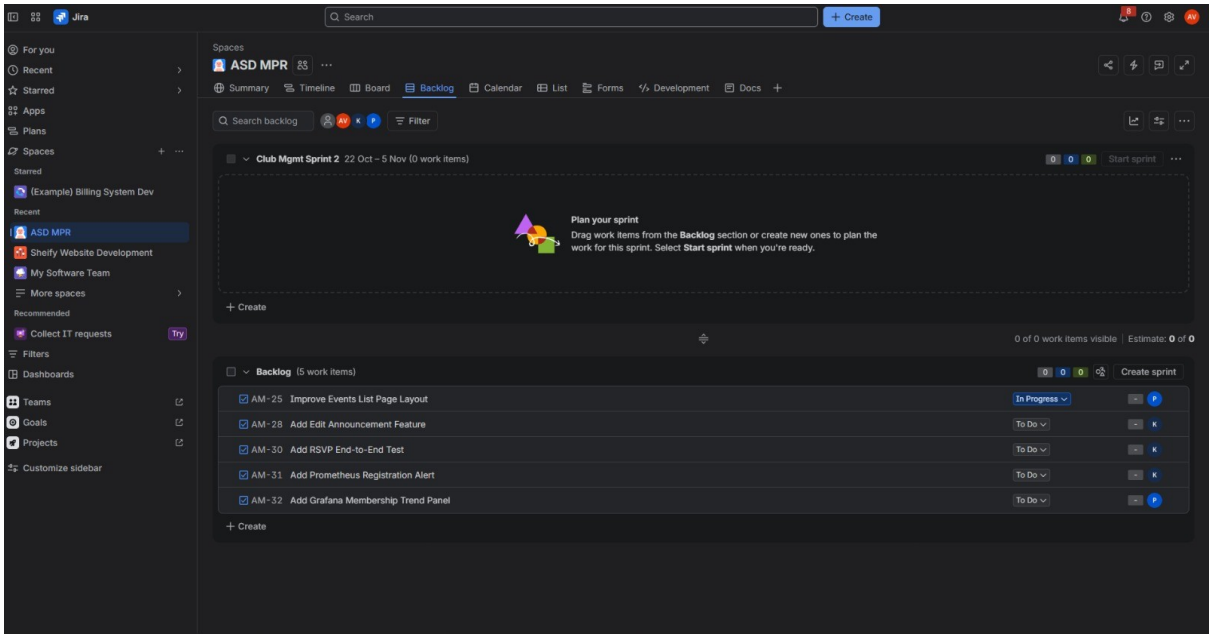


Fig. 3.3: Jira backlog of the ASD MPR project

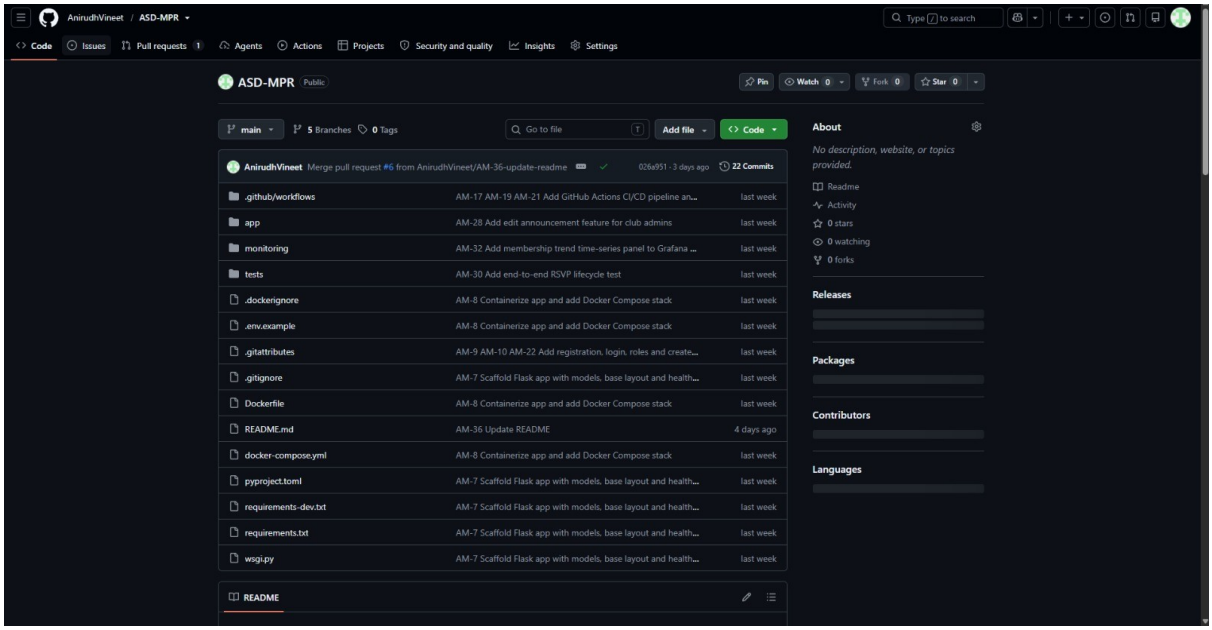


Fig. 3.4: GitHub repository AnirudhVineet/ASD-MPR

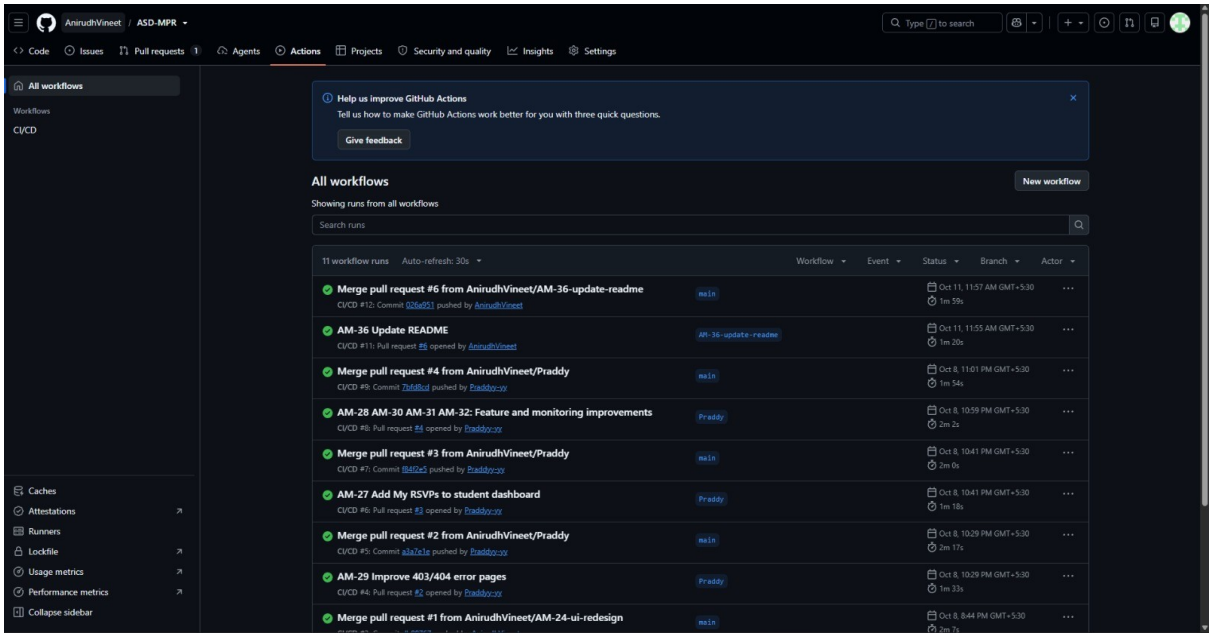


Fig. 3.5: GitHub Actions CI/CD workflow runs

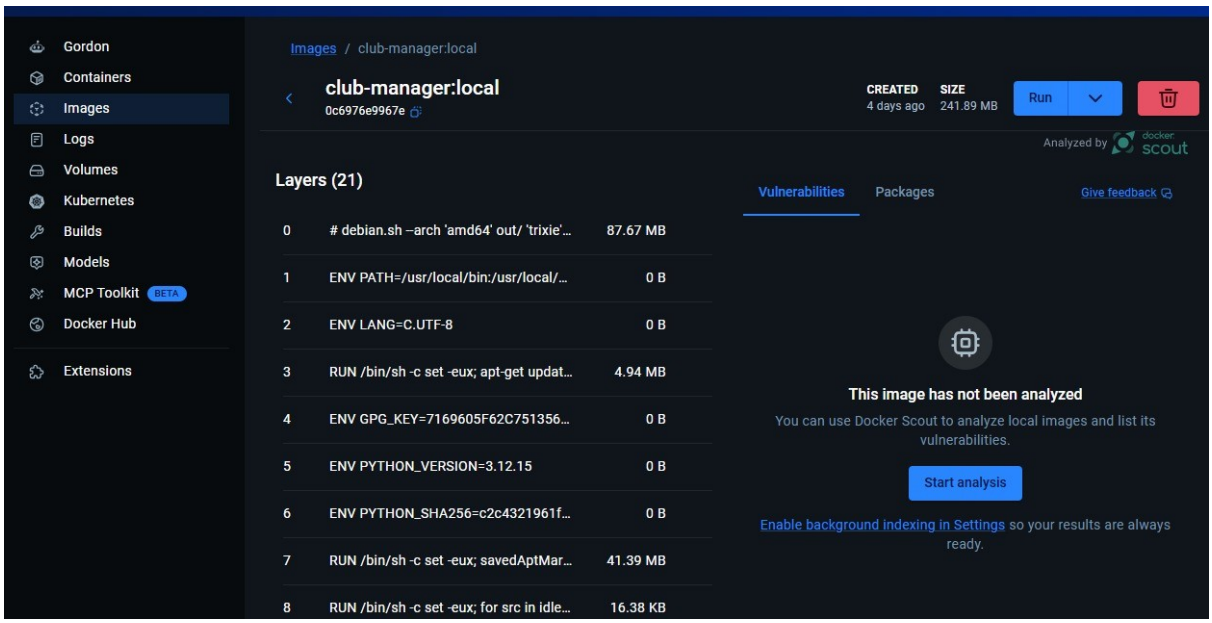


Fig. 3.6: Docker Desktop showing the club-manager:local image

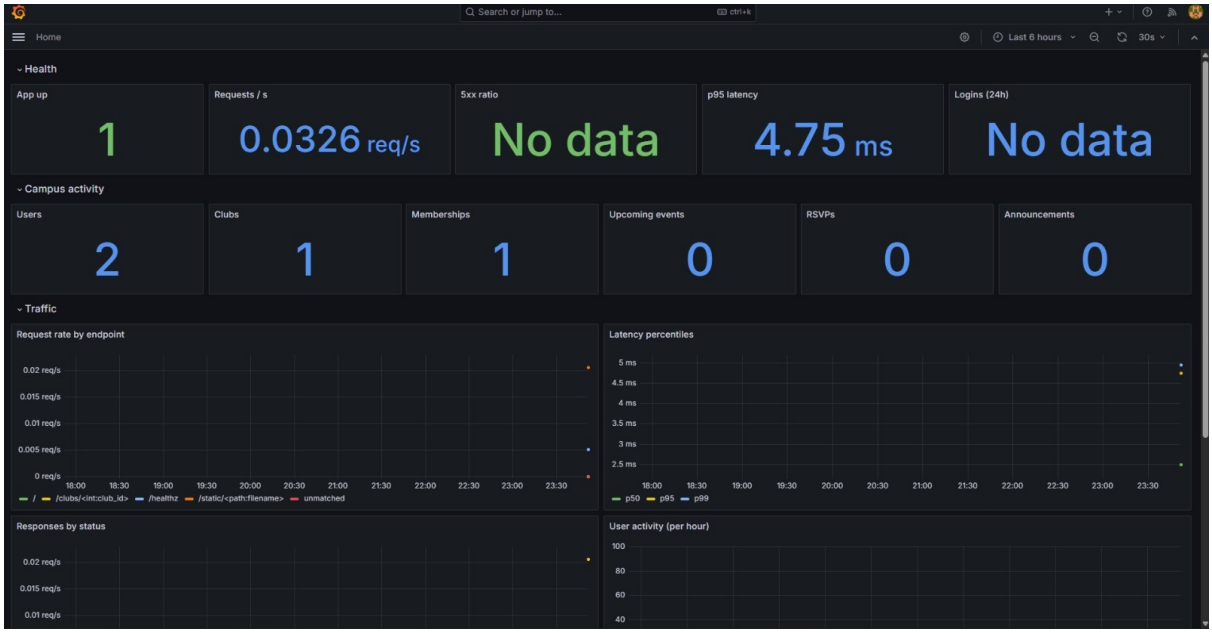


Fig. 3.7: Grafana Club Manager dashboard

3.2 Discussion

Architecture. A single Flask application with blueprints and an application factory keeps the project small enough to test and deploy as one container, while still separating authentication, clubs, events and metrics. Server-rendered templates avoid a separate frontend build and make every page testable through Flask's test client.

Database design. The schema is normalised around Membership and RSVP, and the unique constraints on (user_id, club_id) and (user_id, event_id) stop duplicate memberships and duplicate RSVPs at the database level. Cascades remove a club's memberships, events and announcements together, which a test (test_deleting_club_cascades) checks.

Access control. Because every management route calls the same helpers, the rules in Table 2.4 are applied uniformly, and the tests confirm that non-admin users receive HTTP 403 for management actions and that RSVPs are refused for non-members, past events and full events.

Test results. Table 3.1 lists the 39 tests by module. They were executed on the supplied source for this report with ruff check and pytest; every test passed and ruff reported no findings. The tests use Flask's test client with an in-memory database and cover the behaviour of Sections 2.3 and 2.4; no coverage percentage was measured.

Test module	Tests	Behaviour covered
test_auth.py	6	Registration, duplicate email and short password rejection, login and logout, external ?next ignored, create-admin command
test_clubs.py	13	Club creation and duplicates, student cannot create, directory search and filter, join and leave, member management, club admin can edit but not delete, 404, announcement create, edit and delete
test_events.py	10	Event creation and editing, invalid form, member cannot create, RSVP requires membership, blocked when full, blocked for past events, attendee list, announcements, club-delete cascade, full RSVP lifecycle

Test module	Tests	Behaviour covered
test_main.py	8	Home page, /healthz, 404 and 403 pages, dashboard sections and My RSVPs
test_metrics.py	1	/metrics exposes HTTP and business metrics with bounded endpoint labels
test_templates.py	1	Templates are valid UTF-8
Total	**39**	**All passed** (ruff: no findings)

Table 3.1: Automated test suite, as executed on the supplied source for this report

Monitoring and delivery. Reading the business gauges from the database at scrape time and labelling traffic by URL rule give dashboards that stay accurate and bounded. The smoke-test job is the strongest check in the pipeline because it starts the same Compose stack with PostgreSQL and verifies that Prometheus scrapes the application and that Grafana has loaded the provisioned dashboard.

Actual results and expected benefits. The actual, observed results are the 39 passing tests and clean lint result from the run made for this report, the successful workflow runs visible in the GitHub Actions screenshot, and the metric values shown in the Grafana screenshot. The latency of 4.75 ms was observed at roughly 0.03 requests per second and is not a performance benchmark. Benefits such as easier club administration or better student participation are expected outcomes of the design and were not measured.

Identified Limitations

- **No schema migrations:** tables are created with db.create_all(), so later model changes would need a migration tool.
- **Alerts are not routed:** no Alertmanager is configured, so alert rules are visible only in the Prometheus interface.
- **Single worker:** Gunicorn runs one worker with eight threads so that in-process counters stay consistent; scaling is by running more containers.
- **Time handling:** created_at values use UTC while event start times are compared with the server's local time, and datetime.utcnow() produced deprecation warnings in the test run.
- **Authorisation gaps:** nothing prevents the last club admin from demoting or removing themselves, and site admins can only be created from the command line.
- **RSVP capacity:** the capacity check counts RSVPs in application code and is not protected by a database constraint, so simultaneous requests could in principle exceed the capacity.
- **Test scope:** the pytest suite runs on in-memory SQLite; PostgreSQL is exercised only by the CI smoke test. The /metrics endpoint is public on the application port.

Chapter 4

Conclusion

4.1 Conclusion

Campus Clubs implements a working college club management system: a public club directory, student accounts, club membership, events with capacity-limited RSVPs, announcements, a student dashboard and role-based administration for club and site administrators. The behaviour is backed by a suite of 39 automated tests that all passed in the run made for this report.

The project also demonstrates the Agile and DevOps practices of the subject. Work was broken into Jira work items and traced through branch names, commit messages and pull requests; the application is containerised and started with Docker Compose together with PostgreSQL, Prometheus and Grafana; metrics, a provisioned dashboard and alert rules make the running system observable; and a GitHub Actions pipeline lints, tests, validates configuration, smoke-tests the whole stack and publishes the image on main.

4.2 Future Scope

The following enhancements are proposed future work and are not implemented in the current project.

- **Database migrations:** introduce a migration tool so that schema changes can be applied to existing PostgreSQL data.
- **Alert routing:** add Alertmanager with e-mail or chat notifications for the existing alert rules.
- **Notifications:** send event reminders and announcement e-mails, and add a waiting list for full events.
- **Stronger administration:** protect the last club admin, add an audit log, and add password reset, e-mail verification and login rate limiting.
- **Better testing:** run the test suite against PostgreSQL in CI, report coverage, and add concurrency tests for RSVP capacity.
- **Security scanning:** scan the container image and dependencies for vulnerabilities in the pipeline.

Chapter 5

References

1. **Flask Documentation** – *Python web framework*, https://flask.palletsprojects.com/
2. **SQLAlchemy Documentation** – *The Python SQL toolkit and Object Relational Mapper*, https://docs.sqlalchemy.org/
3. **Flask-SQLAlchemy Documentation** – *SQLAlchemy integration for Flask*, https://flask-sqlalchemy.palletsprojects.com/
4. **Flask-Login and Flask-WTF Documentation** – *User sessions and CSRF-protected forms*, https://flask-login.readthedocs.io/
5. **PostgreSQL Documentation** – *The PostgreSQL relational database*, https://www.postgresql.org/docs/
6. **Gunicorn Documentation** – *Python WSGI HTTP server*, https://docs.gunicorn.org/
7. **Docker Documentation** – *Containerisation platform*, https://docs.docker.com/
8. **Docker Compose Documentation** – *Defining and running multi-container applications*, https://docs.docker.com/compose/
9. **Prometheus Documentation** – *Monitoring system and time series database*, https://prometheus.io/docs/
10. **Prometheus Python Client** – *Instrumenting Python applications*, https://github.com/prometheus/client_python
11. **Grafana Documentation** – *Observability and visualisation platform*, https://grafana.com/docs/
12. **GitHub Actions Documentation** – *Automating build, test and deployment workflows*, https://docs.github.com/en/actions
13. **pytest Documentation** – *The pytest testing framework*, https://docs.pytest.org/
14. **Ruff Documentation** – *An extremely fast Python linter*, https://docs.astral.sh/ruff/
15. **Jira Software Cloud Documentation** – *Agile project tracking*, https://support.atlassian.com/jira-software-cloud/